

Async encode was an ownership redesign.

SYNC

```
encode(item, &mut EncodeBuf)
```

rust

borrow caller buffer
return after body bytes exist



Pending can happen

ASYNC

```
encode(item, EncodeBuffer)
```

rust

```
-> T::Encode
```

move buffer into future
Ready returns EncodeBuffer

Tonic stores

one owned `T::Encode` per in-flight message

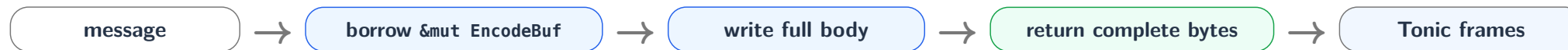
Prost owns

exact field / tag / length / payload resume state inside that future

Sync Encode: Complete Body, Then Frame

Sync Tonic frames only after body bytes are complete and the `&mut EncodeBuf` borrow is gone.

Completed-buffer contract



Code evidence

```
fn encode(&mut self, item, dst: &mut EncodeBuf) -> Result<()>
```

rust

The `&mut` borrow ends when the call returns.

After return

size-check

optional compression

flags + length

yield frame

Tempting but wrong

```
rust
async fn encode(
    &mut self,
    item,
    dst: &mut EncodeBuf,
) -> Result<()>
```

What Tonic must store

in_flight: Option<T::Encode>

one future

per message

Send + 'static

Failure at Pending

caller stack owns &mut borrows



future returns Pending



future still borrows dst / self



cannot store as T::Encode: Send + 'static

Correct storage unit: owned future + owned EncodeBuffer.

encode moves the buffer into a future; Ready returns it for framing.

Encoder API

rust

```
type Encode: Future<Output = Result<EncodeBuffer, Self::Error>>  
+ Send + 'static  
  
fn encode(self: Pin<&mut Self>, item, dst: EncodeBuffer)  
-> Result<Self::Encode, Self::Error>
```

Body driver slot

rust

```
#[pin]  
in_flight: Option<T::Encode>  
in_flight_offset: Option<usize>
```

stored future, not borrowed buffer

Per-message lifecycle

start: move item + buffer in



poll: Pending keeps future stored



ready: buffer returns

One message through EncodedBytes

1. reserve 5-byte header gap



2. move active BytesMut into EncodeBuffer



3. store in_flight = Some(T::Encode)



4. poll Pending / Ready



5. finish compression + header

reserve

```
buf.reserve(HEADER_SIZE)
```

start

```
encoder.encode(item, dst)
```

poll

```
ready!(encode.poll(cx))
```

finish

```
finish_encode_buffer(...)
```

Core point Frame finalization stays in Tonic: compression, max-size check, flags, and gRPC length are written after Ready.

DsaAsyncProstEncode<T>

the object Tonic stores

state

PollEncodeState + payload state

item

Option<T>

sink

DsaProstEncodeSink + buffer

options

length-delimited mode

stage

Start / Body / Done

payload cleanup

descriptor + completion drop path

Pending: keep all compartments



Ready: sink.into_inner() returns EncodeBuffer

Drop order is part of the safety contract: pending payload state may reference item bytes and destination storage.

07 Prost Records the Byte Resume Point

async Tonic encode

Tonic polls one future; Prost remembers the byte-position checkpoint inside it.

Tonic body driver coarse poll slot

stores and polls one `T::Encode`



owned `T::Encode` future per in-flight message

owns message + buffer + offload state

PollEncodeState nested inside the future

byte-level continuation

PollEncodeFrame + payload state

field

index

phase

payload state

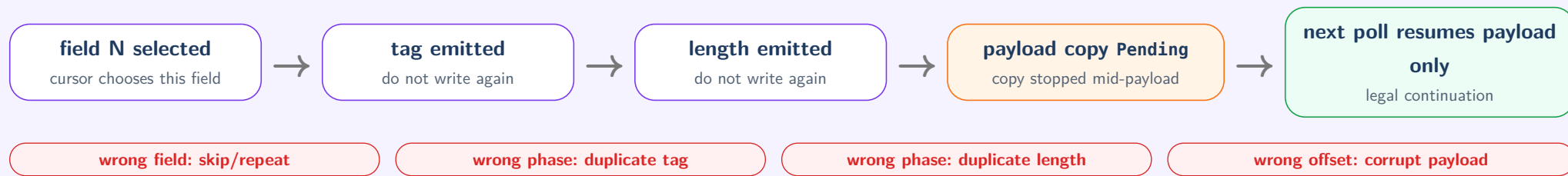
CPU writes keys, lengths, and scalars immediately.

Payload copy may return Pending.

08 One Suspended Length-Delimited Field

async Tonic encode

Checkpoint after payload copy returns Pending



The checkpoint is protocol state: tag and length stay written; only the unfinished payload copy continues.

09 Offload Makes Ownership Physical

async Tonic encode

Pending offload means raw addresses can outlive the encode call.

owned T::Encode holds the physical state

DSA/IAX supplies the pending copy

source bytes: read address stays valid

EncodeBuffer storage: write address stays stable

payload state: descriptor + completion stay
pinned

If ownership is split, a Rust lifetime bug becomes hardware memory corruption.

source valid

payload backing memory stays allocated while hardware may read it

destination stable

EncodeBuffer storage does not move while hardware may write it

payload cleanup drained

payload state's descriptor and completion are owned until Ready or drop cleanup

If an operation may be stored and polled after the call returns, anything needed after suspension must be owned by that operation.

Tonic owns frame boundary

header reservation, compression, size check, flags, length



T::Encode owns suspended work

message, EncodeBuffer, offload state, completion, cancel/drop cleanup



Prost owns wire checkpoint

field, index, phase, and payload-copy continuation

Measurement is downstream

Architecture creates the chance to remove one staged CPU payload copy. Benchmarks decide whether offload submission, wakeup, and resume bookkeeping are smaller.

Async encode = owned resumable state + exact protocol resume points.